Prosjekt 1: Modellering av hjerteslag data I python

Ellay Bryan Beckstrøm

Studentnr: 407891

Oslomet

MAMO1100: Innføring I modellering og beregninger

Introduksjon

I denne prosjektet skal jeg bruke det jeg har lært i MAMO1100 til nå til å lage et program i python som leser inn dataen fra tre forskjellige csv-filer. Disse filene inneholder avstand mellom hvert hjerteslag i en ca 8 timers periode. I fil b og c som skal leses inn så mangler noen pulser og i c det er noen tilfeldige variasjoner i avstanden mellom nabo-hjerteslag. Etter at programmet har lest inn dataen fra de forskjellige csv-filene så skal det lages tre arrays med et array for hver fil. Disse arrayene skal jeg bruke til å lage puls som funksjon av tid, fjerne urealistiske lave pulsverdier fra fil b og c, og tilpasse funksjonen $f(t) = A + Bt + C \sin(Dt)$, til hver av de tre pulsene slik at sammenligne pulsdataene til de tre datasettene og finne forskjeller.

Oppgave 1

Introduksjon til oppgaven

I denne oppgaven så skal jeg lage et program som leser inn informasjon fra en csv fil. Når jeg lager dette programmet så starter jeg først med å kode hele koden for fil a siden den ikke har noen outliers. Denne koden skal jeg så bygge videre på for å passe inn fil b og fil c som har outliers og litt ujevne data.

Puls

For å finne puls og lage en funksjon av tid så har jeg brukt numpy-biblioteket sin cumsum. Det denne gjør er å summere hvert ledd i en array og lage en ny array med de summerte leddene. Når dette er gjort så vil jeg ha avstanden mellom hvert hjerteslag og tiden som går i to arrays hvor jeg bruke tiden som går som x-aksen og avstanden mellom hjerteslag som y-aksen. Siden jeg har en x og en y akse kan jeg bruke disse sammen med matplotlib sin pyplot og lage plott som visualiserer resultatene.

Fjerning av outliers

Til nå så var jeg litt usikker på hvordan jeg skulle fjerne outliers så det jeg gjør først er å skumme igjennom læreboken for å se om jeg fant noe relevant informasjon på fjerning av feildata. Jeg vurderte først å ta gjennomsnittet av alle avstandene men siden de ender opp som en sinus-funksjon og ikke en lineær en så ville ikke dette fynket.

Sinus-funksjon

Måten jeg har tenkt å finne beste funksjon til dataen er å først skrive inn den generelle sinus-funksjon formelen. Etter det så skal jeg bruke curve_fit i python til å tilpasse funksjonen slik at den legger inn verdier som stemmer mest mulig med alle punktene jeg har i csv fil a.

Program

Oppgave 1 a)

Først tok jeg utgangspunkt I «PlotPuls.py» hvor jeg endret og justerte litt på filen slik at jeg kunne få programmet til å kjøre for fil HS7a, så brukte jeg bare copy paste på koden jeg skrev og endret litt på den for fil b og c slik at jeg slapp å kode hele koden for fil b og c.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Tue Oct 21 20:41:39 2025

Oblig 1 Mamo1100

@author: ellaybeckstrom
"""

import matplotlib.pyplot as plt
import numpy as np
from scipy.optimize import curve_fit
#plott av hjerteslag fil a
def plotta():
    # Leser inn data fra csv-fil inn i arrayet "array":
    hs = np.loadtxt('HS7a.csv', delimiter=',')
    #hs1 = np.loadtxt("HS7b.csv", delimiter= ",")
    #hs2 = np.loadtxt("HS7c.csv", delimiter= ",")
    def objective1(x, a, b, d, e):
        return a * np.sin(b*x) + d*x + e #endret på funksjonen fra eksempel plotten slik at jeg får en sinus funksjon som vokser over grunnli

    # curve fitting av funksjon til data
    popt, _ = curve_fit(objective1,np.cumsum(hs),1/hs,[2, 0.02, 1, 00])
    a, b, d, e = popt
    # Skriv ligning til skjerm:
    print('y = %.5f * sin (%.5f x ) + %.5f' % (a, b, d))
    # Definer den rettelinja med disse verdiene for a og b:
    y1 = objective1(np.cumsum(hs),a, b, d, e)
    plott1 = plt.plot(np.cumsum(hs), 1/hs)
    plott2 = plt.plot(np.cumsum(hs), y1, linewidth=2)
    #Gir x og y aksen navn
    plt.xlabel("Tid/sekunder")
    plt.ylabel("Hjerteslag")
    plt.show(plott1, plott2)
plotta()
```

Oppgave 1 b)

Da jeg skulle gjøre oppgave 1.b) så prøvde jeg å finne ut hvordan man fjerner outliers fra ett array ved å skumlese igjennom boken. Etter å ikke ha funnet noe relevant informasjon i pensum boken så spurte jeg chatgpt «How can i remove outliers from an array in python». Hvor chatgpt da viste meg metoden IQR(legger ved bilde under)

1. Using the IQR (Interquartile Range) method

This is one of the most standard and robust methods.

python Copy

```python
import numpy as np

# Example array
data = np.array([1, 2, 3, 4, 5, 100, 6, 7, 8])

# Compute Q1, Q3, and IQR
Q1 = np.percentile(data, 25)
Q3 = np.percentile(data, 75)
IQR = Q3 - Q1

# Define the outlier cutoff
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Filter out outliers
filtered_data = data[(data >= lower_bound) & (da
```

IQR-metoden

IQR metoden (kalt interkvartil avstand på norsk), brukes første og tredje kvartil til å bruke forskjellen mellom disse to kvartilene til å få en verdi som vi ganger med 1,5 (en konstant brukt for metoden) for å lage en øvre og nedre grense for verdier som skal tas med i ett datasett. Ved å bruke denne metoden vil vi fange 95% av verdiene i arrayen hvor de resterende 5% vil være outliers med verdier som ikke passer inn i datasettet.

Måten jeg tenker å implementere denne metoden i koden er ved å bare bruke den på y-verdiene siden x-aksen bare viser verdier for tid og da burde fortsatt tiden i plotten være nøyaktig.

Her er en video jeg brukte til å dypere forstå metoden:

https://www.youtube.com/watch?v=A3gClkblXK8

Ved å se flere videoer så la jeg merke til at jupyterlab nok er enklere for å forstå og kode lange program siden man alltid kan teste og bruke kodene man skriver inn og ikke tester hele programmet på en gang. Jeg kommer nok til å bruke det på neste prosjekt.

Etter rundt 1 time med å ikke få IQR-metoden til å fungere så fikk jeg en rotete kode:

```
def plottb():
    # Leser inn data fra csv-fil inn i arrayet "array":
    hs = np.loadtxt('HS7c.csv', delimiter=',')
    #hs1 = np.loadtxt("HS7b.csv", delimiter= ",")
    #hs2 = np.loadtxt("HS7c.csv", delimiter= ",")
    def objective1(x, a, b, d, e):
        return a * np.sin(b*x) + d*x + e #endret på funksjonen fra eksempel plotten slik at jeg får en sinus funksjon
    # curve fitting av funksjon til data
    popt, _ = curve_fit(objective1,np.cumsum(hs),1/hs,[2, 0.02, 1, 00])
    a, b, d, e = popt
    # Skriv ligning til skjerm:
    print('y = %.5f * sin (%.5f x ) + %.5f' % (a, b, d))
    # Definer den rettelinja med disse verdiene for a og b:
    y1 = objective1(np.cumsum(hs),a, b, d, e)
    #Under her skal jeg legge inn IQR-metoden for å filtrere ut outliers
    Q1 = np.percentile(y1, 25)
    Q3 = np.percentile(y1, 75)

    IQR = Q3 - Q1

    nedre = Q1 - 1.5 * IQR
    øvre = Q3 + 1.5 * IQR
    #
    Q11 = np.percentile(hs, 25)
    Q31 = np.percentile(hs, 75)

    IQR1 = Q31 - Q11

    nedre1 = Q11 - 1.5 * IQR1
    øvre1 = Q31 + 1.5 * IQR1
    hs_invers = 1/hs
    hs_filtrert = hs_invers[(hs_invers >= nedre1) & (hs_invers <= øvre1)]
    #
    #y-arrayen som har blitt filtrert
    y1_filtrert = y1[(y1 >= nedre) & (y1 <= øvre)]
    #y-verdiene som ble filtrert vekk
    #y12 = hs[(hs <= nedre) & (hs >= øvre)]

    plot for filtrert invers
    plt.plot(np.cumsum(hs), hs_filtrert)

    plot for filtrert data, funker
    plt.plot(np.cumsum(hs), y1_filtrert, linewidth=2, )

    plot for ufiltrert data, funker
    plt.plot(np.cumsum(hs), y1, linewidth=1)

    #Gir x og y aksen navn
    plt.xlabel("Tid/sekunder")
    plt.ylabel("Hjerteslag")
    plt.title("Hjerteslag fil b")
    plt.show()
plottb()
```

Da bestemte jeg meg for å begynne på nytt med koden for å bygge den opp igjen på en mer ryddig og effektiv måte. Denne gang legger jeg kun inn det nødvendige for oppgaven i mens jeg bygger opp programmet.

Etter å ha skrevet koden på fikk jeg en ryddigere kode hvor jeg hadde mye bedre forståelse for programmet dette så slik ut:

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.optimize import curve_fit

def plotta():
    #Leser inn data fra csv-fil inn i arrayet "array":
    #Csv filene er tid mellom hver syklus. Hver syklus er når hjertet slår, så da kan vi si at csv-filene har perioden til hjerteslagene.
    #ved å ta den inverse av perioden så får vi frekvens som i dette tilfellet er hjerteslag i sekundet. Dette får vi bruk for senere i oppga
    hs = np.loadtxt('HS7b.csv', delimiter=',')

    #Her definerer jeg funksjonen ved et ekstra ledd "e" slik at sinus-funksjonen vokser over tid
    def objective1(x, a, b, d, e):
        return a * np.sin(b*x) + d*x + e

    #Her brukte jeg curve fit slik som den ble brukt i plot puls
    #verdiene 2, 0.02, 1 og 00 er bare startverdier som curve_fit bruker som utgangspunk
    #ved å ta 1/hs så får jeg frekvensen til hjerteslagene(hvor mye hjerte slår i sekundet)
    popt, _ = curve_fit(objective1,np.cumsum(hs),1/hs,[2, 0.02, 1, 00])
    a, b, d, e = popt

    #Her skrives ligningen ut, men jeg endret på den fra plot puls ved å legge til de ekstra leddene som var i funksjonen
    print('y = %.5f * sin(%.5f * x) + %.5f * x + %.5f' % (a, b, d, e))

    #I denne koden vil jeg bruke np.cumsum for x-aksen siden ved å addere alle leddene fra HS-filene så vil du få en lineær funksjon av tid.
    #her lager jeg y-verdiene i til plotten ved å legge verdier inn i objective1 funksjonen som ble definert tidligere
    #np.cumsum for alle x-verdiene, og legger inn   a, b, d, og e som er verdier tilpasset sinusfunksjonen i popt.
    y1 = objective1(np.cumsum(hs),a, b, d, e)

    #Her definerer jeg x-aksen
    x1 = np.cumsum(hs)

    #plotter den tilpassede sinusfunksjonen Objective 1.
    #plt.plot(x1, y1, linewidth=2, label="Curve fit")
    plt.scatter(x1, y1, linewidth=2, label="Curve fit scatter")

    #i tillegg til å plotte den tilpassede sinusfunksjonen så plotter jeg frekvensen til hjerteslag ved å ta den inverse av hvert hjerteslag
    #jeg gjør dette slik at jeg kan se hvor nøyaktig curve_fit har laget grafen.
    #plt.plot(x1, 1/hs, linewidth=1, label="Invers")
    plt.scatter(x1, 1/hs, linewidth=1, label="Invers scatter")

    #bruker plt.legend til å lage grids og å gi mer informasjon ut ifra plotten.
    plt.legend(); plt.grid(True, which="both", linestyle="--", alpha=0.4)

    #Gir plotten tittel og legger inn navn på aksene.
    plt.title("HS7a fil")
    plt.xlabel("Tid/sekunder")
    plt.ylabel("puls")

    #Viser plotten
    plt.show()

plotta()
```

Nå som jeg har utgangspunktet til koden bygger jeg om på den ved å prøve meg på IQR-metoden igjen, men denne gang isolerer jeg først hele IQR-koden og så kopierer jeg den inn i koden slik at jeg bare må endre på noen få variabler for å fjerne outliers fra koden. Etter litt tenking så kom jeg frem til at jeg kan bruke IQR-metoden på frekvensen(hs/1) av hjerteslagene siden frekvensen til hjerteslagene varierer mellom 50 og 70 hjerteslag i minuttet så da trengs kun de under og over dette å fjernes.

Her er den nye IQR-koden jeg skrev. Denne koden fungerte perfekt til å filtrere vekk y-verdiene, men da ble det ett problem og det var at den filtrerte y-arrayen hadde mindre ledd enn det x-arrayen hadde og da klarte ikke python og plotte dem sammen.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Thu Oct 23 14:53:08 2025

IQR-metoden for å fjerne outliers.

@author: ellaybeckstrom
"""
import numpy as np
import matplotlib.pyplot as plt


hs = np.loadtxt("H57b.csv", delimiter=",")

x1 = np.cumsum(hs)

yi = 1/hs

Q1 = np.percentile(yi, 25)
Q3 = np.percentile(yi, 75)

IQR = Q3 - Q1

nedre = Q1 - 1.5 * IQR
øvre = Q3 + 1.5 * IQR
#y-arrayen som har blitt filtrert
yi_f = yi[(yi >= nedre) & (yi <= øvre)]
#y-verdiene som ble filtrert vekk
yi_f1 = yi[(yi <= nedre) | (yi >= øvre)]

print("puls filtrert " , yi_f)
print("puls filtrert vekk ", yi_f1)


plt.scatter(x1, yi, label="ikke filtrert")
plt.show()

plt.scatter(x1, yi_f, label="filtrert")
plt.show()
```

Mask i python

Det jeg gjor da var å spørre chatgpt om hvordan jeg kan fikse dette problemet, og da kom den opp med en løsning der jeg bruker «mask» i python. Mask i python gjør slik at hver verdi i arrayen fra den y-filtrert blir boolsk hvor hver verdi innenfor denne arrayen får en true eller false verdi basert på om de er filtrert vekk(false) eller om de ikke er filtrert vekk(true). Når jeg da bruker mask på x og y, så vil alle true-verdiene i y-arrayen bli tatt med i x-arrayen og alle false-verdiene i y-arrayen blir fjernet i x-arrayen.

Ved å bruke dette så forsvant mye av tiden i x-aksen da ble jeg nødt til å prøve med en annen metode.

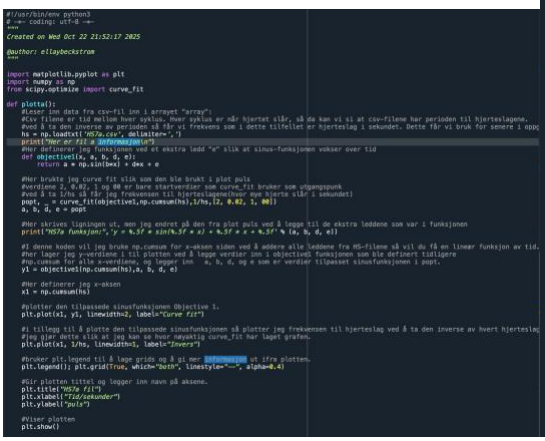
Linspace for x-verdier

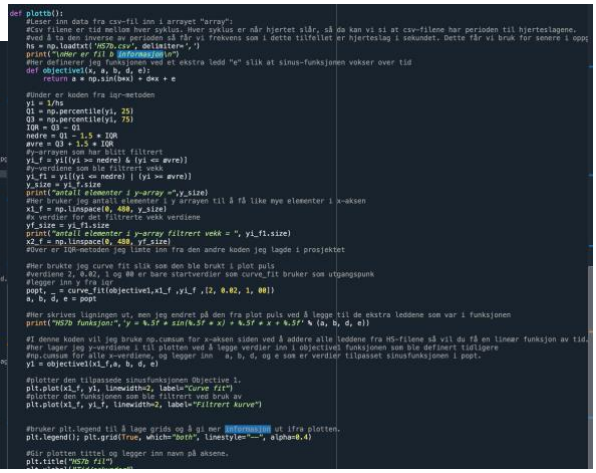
Da fikk jeg en tanke om at jeg kan bruke linspace til å lage en array som går fra 0 til 480 hvor mengden ledd er antall ledd i den filtrerte y-aksen. Da trengte jeg bare å telle antall ledd i y-aksen slik at jeg kunne bruke disse verdiene i linspace slik np.linspace(0, 480, y.size).

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Thu Oct 23 14:53:08 2025

IQR-metoden for å fjerne outliers.

@author: ellaybeckstrom
"""
import numpy as np
import matplotlib.pyplot as plt


hs = np.loadtxt("HS7c.csv", delimiter=",")

x1 = np.cumsum(hs)

yi = 1/hs

Q1 = np.percentile(yi, 25)
Q3 = np.percentile(yi, 75)

IQR = Q3 - Q1

nedre = Q1 - 1.5 * IQR
øvre = Q3 + 1.5 * IQR
#y-arrayen som har blitt filtrert
yi_f = yi[(yi >= nedre) & (yi <= øvre)]
#y-verdiene som ble filtrert vekk
yi_f1 = yi[(yi <= nedre) | (yi >= øvre)]



print("puls filtrert " , yi_f)
print("puls filtrert vekk ", yi_f1)

y_size = yi_f.size
print("antall elementer i y-array =",y_size)
#Her bruker jeg antall elementer i y arrayen til å få like mye elementer i x-aksen
x1_f = np.linspace(0, 480, y_size)
#x verdier for det filtrerte vekk verdiene
yf_size = yi_f1.size
print("antall elementer i y-array filtrert vekk")
x2_f = np.linspace(0, 480, yf_size)
```

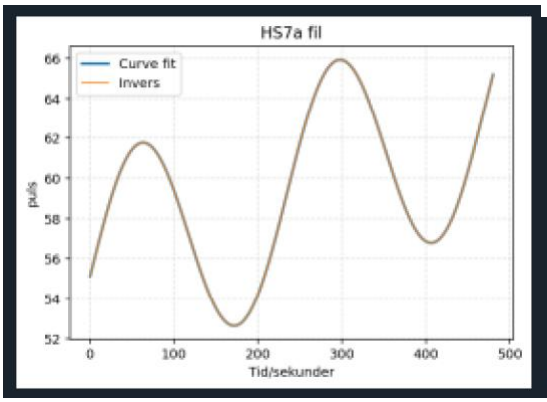
Da fikk jeg denne koden som filtrerte vekk outliers fra koden. Så var det bare å lime denne inn i hoved koden slik at jeg får kodene til å kjøre som en.

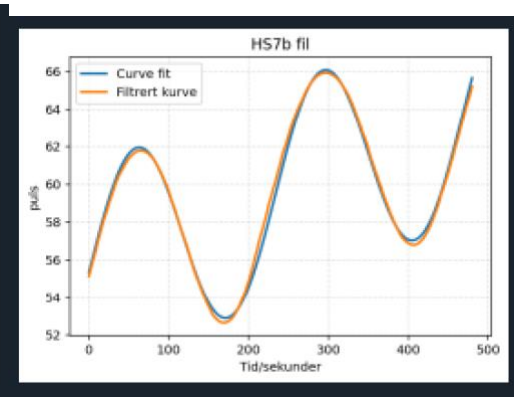
Etter at jeg skrev om på hovedkoden så ser den slik ut.





Av å kjøre denne koden så får jeg disse resultatene





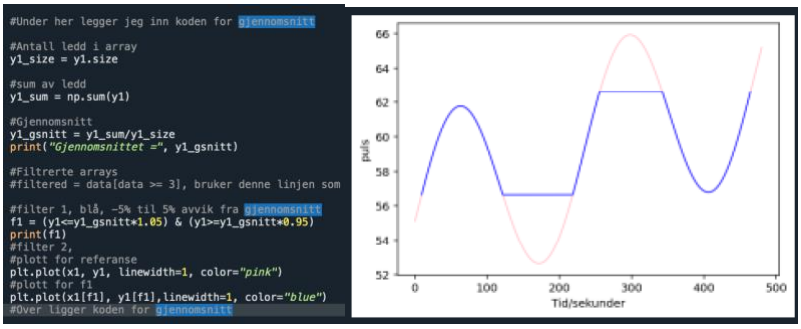

```
Her er fil a informasjon

HS7a funksjon: y = 5.56858 * sin(0.02677 * x) + 0.01771 * x +
55.12723

Her er fil b informasjon

antall elementer i y-array = 18067
antall elementer i y-array filtrert vekk =  4635
HS7b funksjon: y = 5.52057 * sin(0.02687 * x) + 0.01770 * x +
55.34466
```

Oppgave 2)

For å markere de forskjellige delene av funksjonene så tenkte jeg å ta gjennomsnittlig preprossesert puls over hele 8 timers perioden og lage 4 forskjellig plotter med 1 for hver del av gjennomsnittet som skal vises, slik at alle delene kan få forskjellige farger.

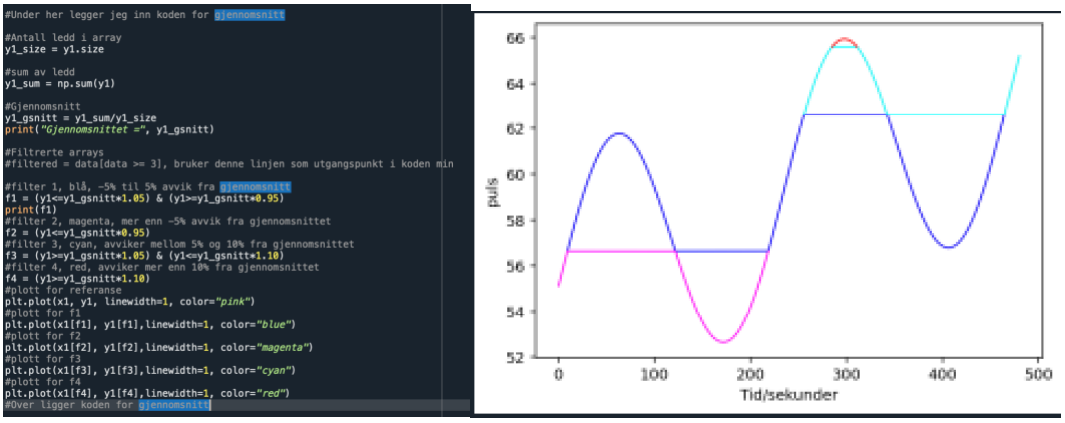
For å gjennomføre dette så tenker jeg først å bruke «y.size» for å finne ut hvor mange ledd det er i arrayen, så addere alle leddene i arrayen ved å bruke «np.sum» slik at jeg kan bruke formelen for

gjennomsnitt (sum av alle element/antall element). Så må jeg lage 4 arrays hvor hver array passer inn i en funksjon som oppfyller kriteriet til oppgaven. Gjennom å gjøre mye feil kom jeg endelig frem til en kode som jeg kan utvikle videre til å løse oppgaven.



Her bruker jeg «mask» konseptet til å filtrere ut verdier fra både x og y aksen slik at det blir like mye på begge sider.

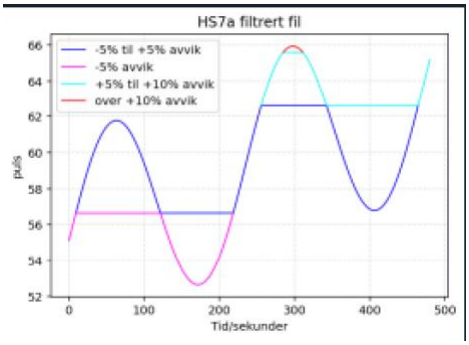
Etter den var ferdig kodet så den slik ut.



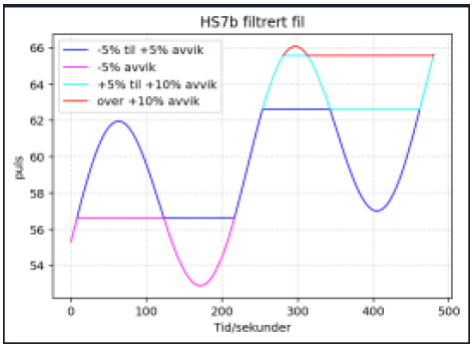
Så var jeg bare nødt til å få denne gjennomsnittskoden inn i hovedkoden.

Etter at dette var gjort fikk jeg disse grafene i hovedkoden.

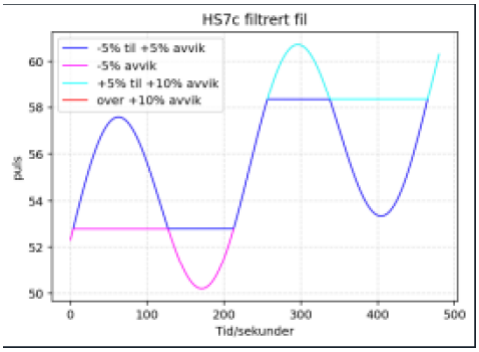
Fil a informasjon



Fil b informasjon



Fil c informasjon



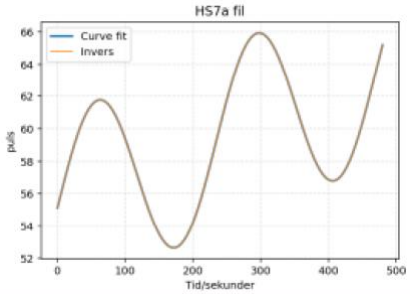
Oppgave 3 a)

Etter å ha kodet ferdig koden for HS7a filen så la jeg merke til at jeg hadde løst første del av oppgave 3 a) ved å følge koden fra plot.puls og justere funksjonen ved å gange d med x og addere ett ekstra ledd «e» slik at sinusfunksjonen stiger over likevektslinjen. Så måtte jeg bare legge inn de preprosseserte arrayene fra oppgave 1 b) inn i curve_fit slik at sinus-funksjonen ble tilpasset de preprosseserte verdiene. Så plotten jeg dem sammen med den filtrerte funksjonen. I fil a så ble curve_fit funksjonen lik som den inverse funksjonen av puls. Men i fil b og c så var det litt forskjell i resultatene mann fikk fra den filtrerte funksjonen og curve_fit funksjonen.

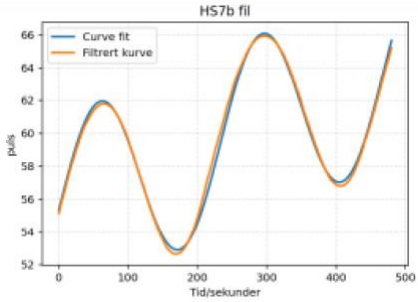
Oppgave 3 b)

Under ligger alle funksjonene jeg skal sammenligne

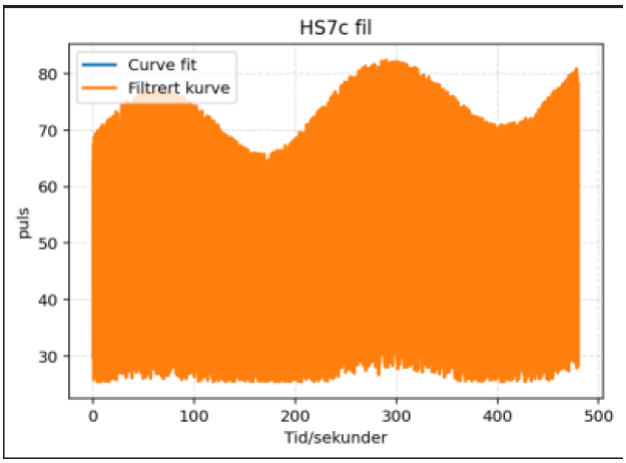
Fil a



Fil b



Fil c



Under så ser vi verdiene til alle funksjonene

Vi ser at det er litt forskjell mellom alle funksjonene. Det er siden alle funksjonene er i grunn basert på csv-filen de leser. Fil a er den «mest» nøyaktige siden den ikke har noen feil verdier eller verdier som mangler, mens b er veldig lik a, men ikke nøyaktig lik siden det er andre verdier og det er verdier har blitt filtrert vekk. Fil c mye mer ujevn informasjon som gjør at programmet får større endring i verdiene ettersom informasjonen den har blitt tilgitt er mer misinformerende.

```
In [68]: %runfile '/Users/ellaybeckstrom/Library/CloudStorage/OneDrive-OsloMet/Skole/Modellering/
1.semester/Innleveringer/Prosjekt 1/forsøk2.py' --wdir
Her er fil a informasjon

HS7a funksjon: y = 5.56858 * sin(0.02677 * x) + 0.01771 * x + 55.12723
Gjennomsnittet = 59.63360349659229
[False False False ... False False False]

Her er fil b informasjon

antall elementer i y-array = 18067
antall elementer i y-array filtrert vekk =  4635
HS7b funksjon: y = 5.52057 * sin(0.02687 * x) + 0.01770 * x + 55.34466
Gjennomsnittet = 59.61677369691802
[False False False ... False False False]

Her er fil c informasjon

antall elementer i y-array = 21569
antall elementer i y-array filtrert vekk =  1150
HS7c funksjon: y = 4.45180 * sin(0.02691 * x) + 0.01346 * x + 52.31678
Gjennomsnittet = 55.56908684216784
[False False False ... False False False]
```